

EXP NO: 1

Date:27-8-26

CLUSTERING WITH K-MEANS AND DIMENSIONALITY REDUCTION

WITH PCA

AIM:

To demonstrate the application of unsupervised learning models, specifically **K-Means Clustering** for grouping data points and **Principal Component Analysis (PCA)** for dimensionality reduction and visualization using a suitable dataset.

ALGORITHM:

1. K-Means Clustering

1. Generate or load the dataset.
2. Choose the number of clusters, **K**.
3. Initialize K cluster centroids.
4. Assign each data point to the nearest centroid using Euclidean distance.
5. Recalculate the centroids using the mean of the points in each cluster.
6. Repeat the assignment and update steps until the centroids become stable or the maximum number of iterations is reached.
7. Use the **Elbow Method** to determine the optimal value of K.
8. Calculate the **Silhouette Score** to evaluate the clustering quality.

2. Principal Component Analysis (PCA)

1. Generate or load a high-dimensional dataset.
2. Standardize the features so that they have mean 0 and variance 1.
3. Calculate the covariance matrix.
4. Calculate the eigenvalues and eigenvectors.
5. Sort the principal components according to decreasing eigenvalues.
6. Select the required number of principal components.
7. Project the original dataset onto the selected components.
8. Visualize the reduced dataset using the principal components.
9. Apply K-Means clustering on the PCA-reduced data and calculate its Silhouette Score.

CODE:

```
app.py > ...
1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5
6  from sklearn.datasets import make_blobs
7  from sklearn.preprocessing import StandardScaler
8  from sklearn.cluster import KMeans
9  from sklearn.decomposition import PCA
10 from sklearn.metrics import silhouette_score
11
12 # -----
13 # Part 1: K-Means Clustering
14 # -----
15 print("--- Part 1: K-Means Clustering ---")
16
17 # 1. Generate dataset
18 X, y = make_blobs(
19     n_samples=300,
20     centers=3,
21     cluster_std=0.60,
22     random_state=42
23 )
24
25 df_kmeans = pd.DataFrame(
26     X,
27     columns=['Feature_1', 'Feature_2']
28 )
29
30 print("\nOriginal K-Means Dataset Head:")
31 print(df_kmeans.head())
32
33 # 2. Elbow Method
34 wcss = []
35
36 for i in range(1, 11):
37     kmeans = KMeans(
38         n_clusters=i,
39         init='k-means++',
40         max_iter=300,
41         n_init=10,
42         random_state=42
43     )
44
45     kmeans.fit(X)
46     wcss.append(kmeans.inertia_)
47
48 plt.figure(figsize=(10, 6))
49 plt.plot(
50     range(1, 11),
```

```

app.py > ...
47
48 plt.figure(figsize=(10, 6))
49 plt.plot(
50     range(1, 11),
51     wcss,
52     marker='o',
53     linestyle='--'
54 )
55
56 plt.title('Elbow Method for Optimal K (K-Means)')
57 plt.xlabel('Number of Clusters (K)')
58 plt.ylabel('WCSS')
59 plt.grid(True)
60 plt.show()
61
62 # 3. Apply K-Means with chosen K
63 optimal_k = 3
64
65 kmeans = KMeans(
66     n_clusters=optimal_k,
67     init='k-means++',
68     max_iter=300,
69     n_init=10,
70     random_state=42
71 )
72
73 clusters = kmeans.fit_predict(X)
74 df_kmeans['Cluster'] = clusters
75
76 # 4. Visualize K-Means clusters
77 plt.figure(figsize=(10, 8))
78
79 sns.scatterplot(
80     x='Feature_1',
81     y='Feature_2',
82     hue='Cluster',
83     data=df_kmeans,
84     palette='viridis',
85     s=100,
86     alpha=0.8
87 )
88
89 plt.scatter(
90     kmeans.cluster_centers_[:, 0],
91     kmeans.cluster_centers_[:, 1],
92     s=300,
93     c='red',
94     marker='X',
95     label='Centroids'
96 )
97

```

```

app.py > ...
96 |
97
98 plt.title(f'K-Means Clustering with K={optimal_k}')
99 plt.xlabel('Feature 1')
100 plt.ylabel('Feature 2')
101 plt.legend()
102 plt.grid(True)
103 plt.show()
104
105 # 5. Silhouette Score
106 silhouette_avg = silhouette_score(X, clusters)
107
108 print(
109     f"\nSilhouette Score for K-Means "
110     f"(K={optimal_k}): {silhouette_avg:.3f}"
111 )
112
113 # -----
114 # Part 2: PCA
115 # -----
116 print("\n--- Part 2: Dimensionality Reduction with PCA ---")
117
118 # 1. Generate 4D dataset
119 X_pca, y_pca = make_blobs(
120     n_samples=500,
121     n_features=4,
122     centers=4,
123     cluster_std=1.0,
124     random_state=25
125 )
126
127 df_pca_original = pd.DataFrame(
128     X_pca,
129     columns=[f'Feature_{i+1}' for i in range(X_pca.shape[1])]
130 )
131
132 df_pca_original['True_Cluster'] = y_pca
133
134 print("\nOriginal PCA Dataset Head:")
135 print(df_pca_original.head())
136
137 print(
138     f"Original PCA Dataset Shape: "
139     f"{df_pca_original.shape}"
140 )
141
142 # 2. Standardize
143 scaler = StandardScaler()
144
145 X_pca_scaled = scaler.fit_transform(X_pca)
146

```

```
app.py > ...
143 scaler = StandardScaler()
144
145 X_pca_scaled = scaler.fit_transform(X_pca)
146
147 # 3. PCA (4D -> 2D)
148 pca = PCA(n_components=2)
149
150 principal_components = pca.fit_transform(
151     X_pca_scaled
152 )
153
154 df_principal_components = pd.DataFrame(
155     principal_components,
156     columns=[
157         'Principal_Component_1',
158         'Principal_Component_2'
159     ]
160 )
161
162 df_principal_components['True_Cluster'] = y_pca
163
164 explained_variance = pca.explained_variance_ratio_
165
166 print("\nPrincipal Components Head:")
167 print(df_principal_components.head())
168
169 print(
170     f"\nExplained Variance Ratio: "
171     f"{explained_variance}"
172 )
173
174 print(
175     f"Total Explained Variance by 2 PCs: "
176     f"{explained_variance.sum():.3f}"
177 )
178
179 # 4. Visualize PCA result
180 plt.figure(figsize=(10, 8))
181
182 sns.scatterplot(
183     x='Principal_Component_1',
184     y='Principal_Component_2',
185     hue='True_Cluster',
186     data=df_principal_components,
187     palette='Paired',
188     s=100,
189     alpha=0.8
190 )
191
192 plt.title(
```

```

app.py > ...
192 plt.title(
193     'PCA - Dimensionality Reduction to 2 Components'
194 )
195
196 plt.xlabel(
197     f'PC1 ({explained_variance[0]*100:.2f}%)'
198 )
199
200 plt.ylabel(
201     f'PC2 ({explained_variance[1]*100:.2f}%)'
202 )
203
204 plt.grid(True)
205 plt.show()
206
207 # 5. K-Means on PCA-reduced data
208 kmeans_pca = KMeans(
209     n_clusters=4,
210     init='k-means++',
211     max_iter=300,
212     n_init=10,
213     random_state=42
214 )
215
216 clusters_pca = kmeans_pca.fit_predict(
217     principal_components
218 )
219
220 df_principal_components[
221     'KMeans_Cluster_on_PCA'
222 ] = clusters_pca
223
224 plt.figure(figsize=(10, 8))
225
226 sns.scatterplot(
227     x='Principal_Component_1',
228     y='Principal_Component_2',
229     hue='KMeans_Cluster_on_PCA',
230     data=df_principal_components,
231     palette='viridis',
232     s=100,
233     alpha=0.8
234 )
235
236 plt.scatter(
237     kmeans_pca.cluster_centers_[0],
238     kmeans_pca.cluster_centers_[1],
239     s=300,
240     c='red',
241     marker='X',

```

```

235
236 plt.scatter(
237     kmeans_pca.cluster_centers[:, 0],
238     kmeans_pca.cluster_centers[:, 1],
239     s=300,
240     c='red',
241     marker='X',
242     label='Centroids'
243 )
244
245 plt.title(
246     'K-Means Clustering on PCA-Reduced Data'
247 )
248
249 plt.xlabel('Principal Component 1')
250 plt.ylabel('Principal Component 2')
251 plt.legend()
252 plt.grid(True)
253 plt.show()
254
255 # 6. Silhouette Score
256 silhouette_avg_pca = silhouette_score(
257     principal_components,
258     clusters_pca
259 )
260
261 print(
262     f"\nSilhouette Score for K-Means "
263     f"on PCA-Reduced Data (K=4): "
264     f"{silhouette_avg_pca:.3f}"
265 )

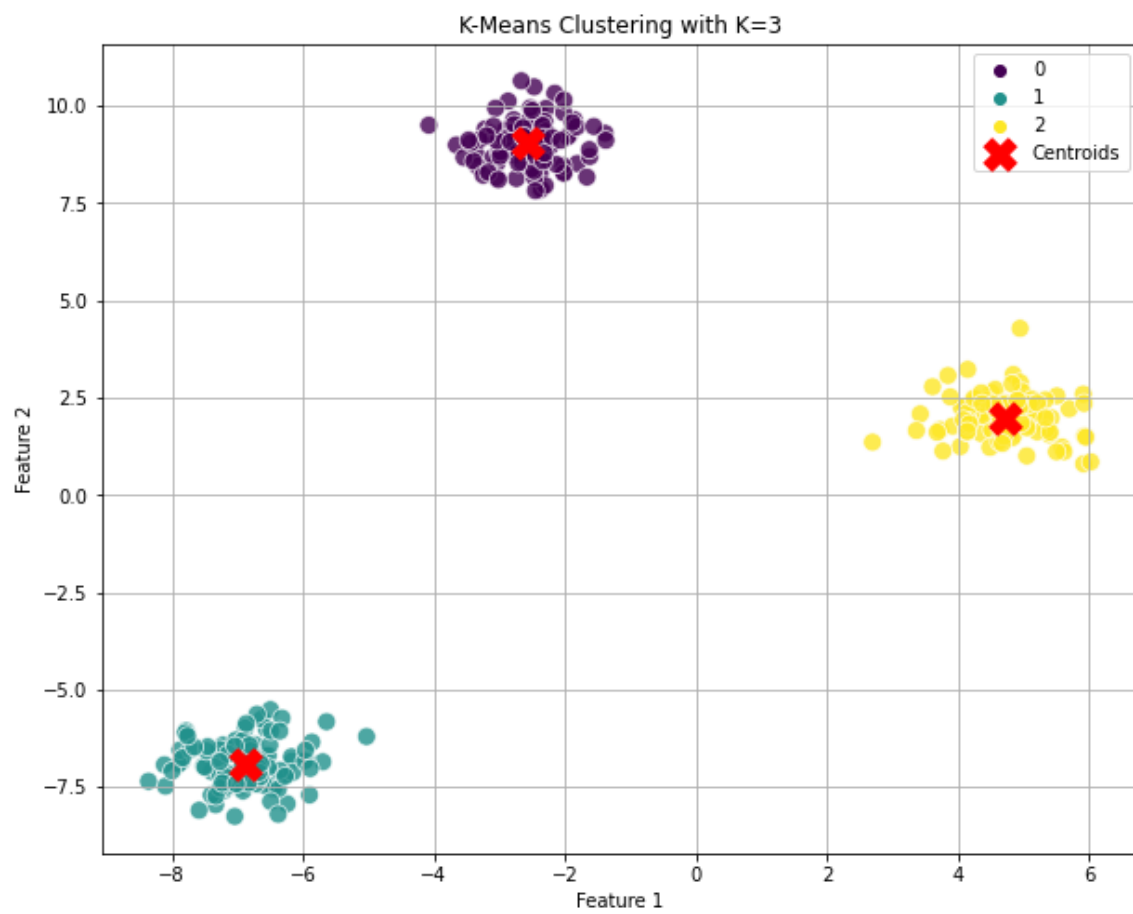
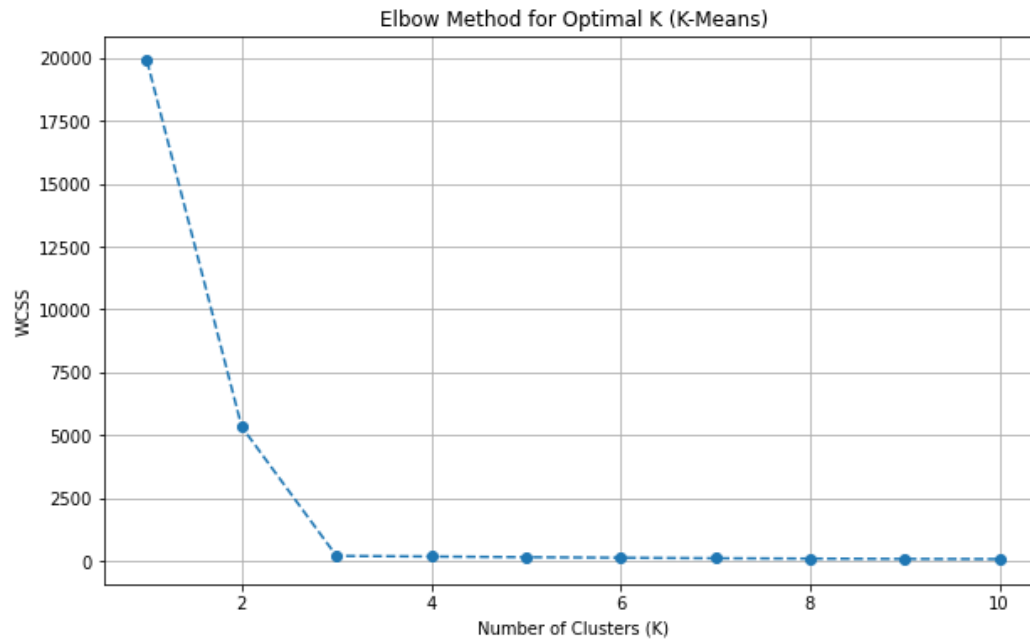
```

OUTPUT:

--- Part 1: K-Means Clustering ---

Original K-Means Dataset Head:

	Feature_1	Feature_2
0	-7.155244	-7.390016
1	-7.395875	-7.110843
2	-2.015671	8.281780
3	4.509270	2.632436
4	-8.102502	-7.484961

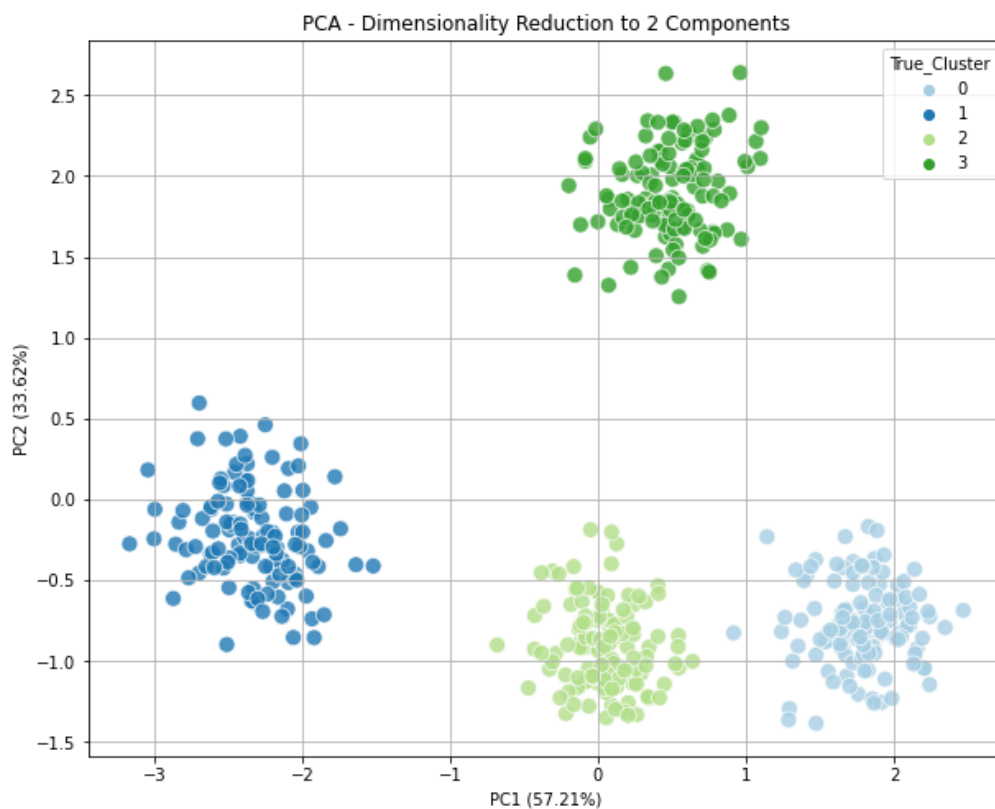


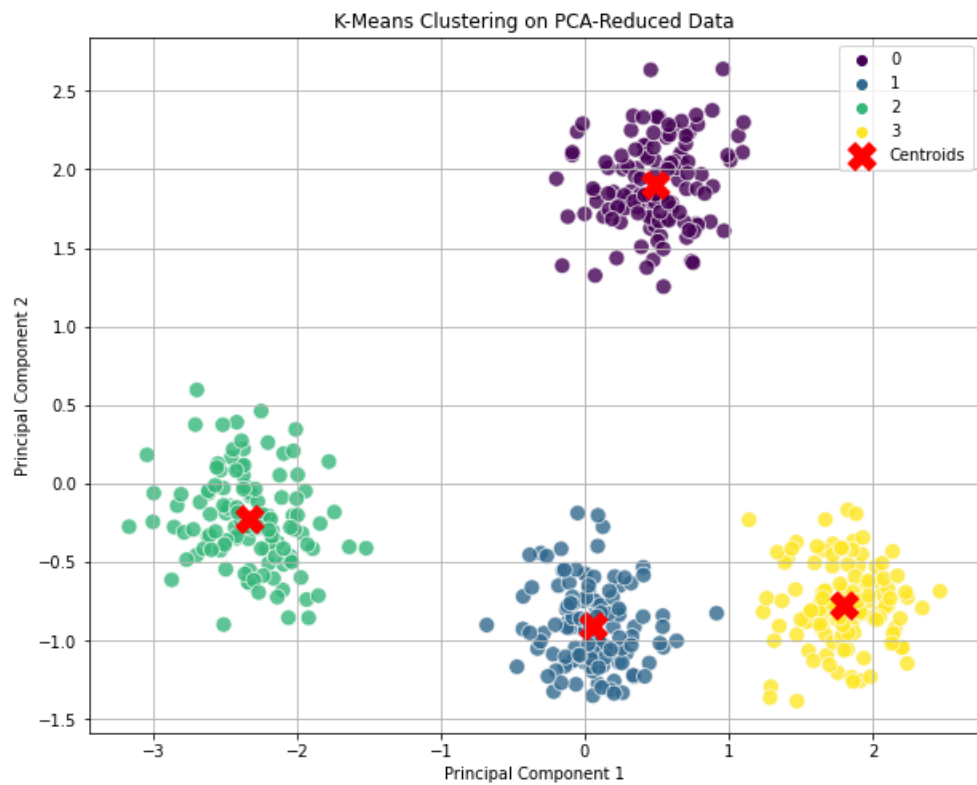
--- Part 2: Dimensionality Reduction with PCA ---

Original PCA Dataset Head:

	Feature_1	Feature_2	Feature_3	Feature_4	True_Cluster
0	-0.638667	1.110057	-6.400722	-0.204990	3
1	-2.951556	-7.657445	3.844794	0.903589	1
2	-0.253177	2.125103	-7.869801	0.559678	3
3	-2.151209	3.401400	-5.734930	0.965230	3
4	-2.347519	-7.230467	3.478891	-0.443440	1

Original PCA Dataset Shape: (500, 5)





RESULT:

The **K-Means Clustering** and **Principal Component Analysis (PCA)** techniques were successfully implemented.